

DD2424 - Assignment 2

David Marzban

dmarzban@kth.se

Update Maj 9, 2026

1. Instruction

In this assignment, a two-layer neural network was trained and tested on the CIFAR-10 dataset for image classification. The implementation builds on Assignment 1, with extensions including a hidden layer, modified forward pass and backward pass, and the use of cyclical learning rates.

The work also includes gradient verification, hyperparameter tuning for the regularization parameter λ , and evaluation of model performance on training, validation and test datasets.

2. The following content

2.1 Analytic gradient computations:

To verify the correctness of the analytical gradients, they were compared with gradients computed using PyTorch and following the derivations provided in assignment 2 and lecture 4 material.

I implemented the backward pass manually using backpropagation and computed gradient using my own implementation (BackwardPass) function and PyTorch automatic differentiation (ComputeGradsWithTorch) function. A small subset of the training data (100 samples) and a reduced network were used to make the computation efficient.

The absolute difference between the gradients were measured for all parameters ($W1$, $b1$, $W2$, $b2$). The maximum absolute errors obtained were:

Max abs diff $W1$: 2.42861e-17	=>	$W1 = 2.43 * 10^{-17}$
Max abs diff $W2$: 7.63278e-17	=>	$W2 = 7.63 * 10^{-17}$
Max abs diff $b1$: 1.21431e-17	=>	$b1 = 1.21 * 10^{-17}$
Max abs diff $b2$: 1.38778e-17	=>	$b2 = 1.39 * 10^{-17}$

The differences are extremely small and are in the order of machine precision, indicating that the analytical gradients implementation is correct and free of bugs.

2.2 The curves for the training and validation loss/cost (cyclical learning rates):

To evaluate the effect of cyclical learning rates, the network was trained using cyclical learning rates with default parameter settings. Training and validation loss and cost were recorded multiple times during each cycle in figures 1 and 2 replicate figures 3 and 4 from assignment.

The results show that both the training and validation loss decrease over time, indicating that the network is learning. Due to the cyclical learning rate, the curves

aren't strictly monotonic but instead exhibit oscillations. The validation curves closely follow the training curves, with slightly higher value which is expected.

The parameter used in figure 1 give the accuracy of test data was 45.81% and the parameter used in figure 2 give the accuracy of test data was 47.46%. The results demonstrate that increasing the number of cycles improves mode performance.

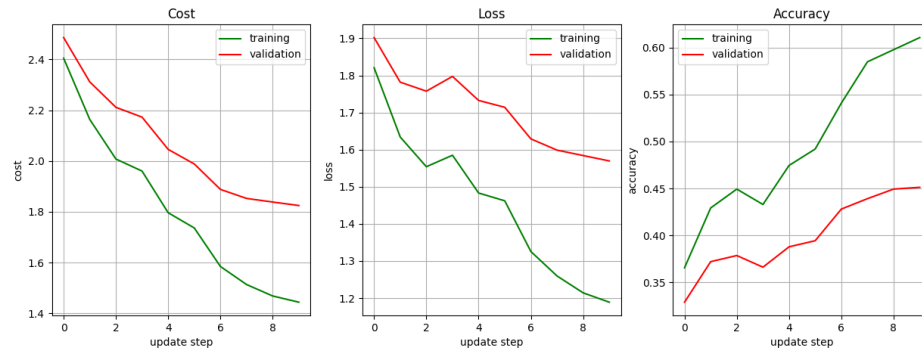


Figure 1: Training and validation loss, cost and accuracy with parameters setting n_s : 500, $\eta_{\min}$: $1e-5$, $\eta_{\max}$: $1e-1$, n_{batch} : 100, $\lambda = 0.01$ and n_{epochs} :10 (means one cycle)

Result figure 1:

$t=100$, $\eta=0.0198$, $\text{loss}=1.8207$, $\text{cost}=2.4047$, $\text{acc}=0.3655$
 $t=200$, $\eta=0.0398$, $\text{loss}=1.6340$, $\text{cost}=2.1636$, $\text{acc}=0.4292$
 $t=300$, $\eta=0.0598$, $\text{loss}=1.5540$, $\text{cost}=2.0078$, $\text{acc}=0.4493$
 $t=400$, $\eta=0.0798$, $\text{loss}=1.5849$, $\text{cost}=1.9603$, $\text{acc}=0.4329$
 $t=500$, $\eta=0.0998$, $\text{loss}=1.4831$, $\text{cost}=1.7964$, $\text{acc}=0.4745$
 $t=600$, $\eta=0.0802$, $\text{loss}=1.4621$, $\text{cost}=1.7363$, $\text{acc}=0.4920$
 $t=700$, $\eta=0.0602$, $\text{loss}=1.3254$, $\text{cost}=1.5847$, $\text{acc}=0.5409$
 $t=800$, $\eta=0.0402$, $\text{loss}=1.2597$, $\text{cost}=1.5137$, $\text{acc}=0.5849$
 $t=900$, $\eta=0.0202$, $\text{loss}=1.2142$, $\text{cost}=1.4686$, $\text{acc}=0.5978$
 $t=1000$, $\eta=0.0002$, $\text{loss}=1.1890$, $\text{cost}=1.4442$, $\text{acc}=0.6107$

Final test accuracy 0.4581 => 45.81%

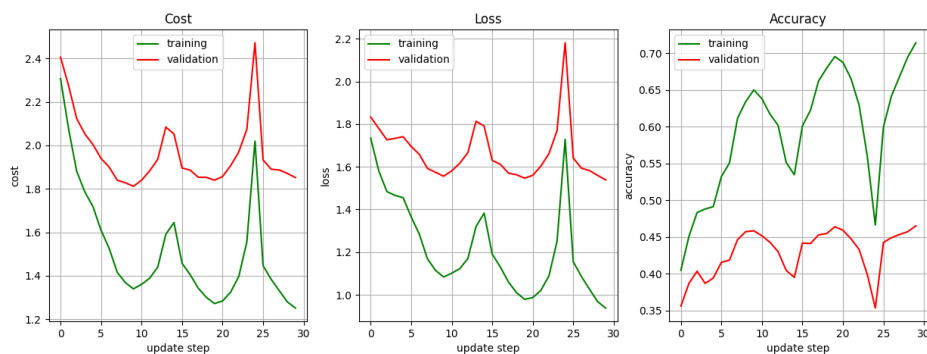


Figure 2: Training and validation loss, cost and accuracy with parameters setting n_s : 800, $\eta_{\min}$: $1e-5$, $\eta_{\max}$: $1e-1$, n_{batch} : 100, $\lambda = 0.01$ and n_{epochs} :48 (means three cycles)

Result figure 2:

t=160, eta=0.0199, loss=1.7347, cost=2.3077 ,acc=0.4046
t=320, eta=0.0399, loss=1.5811, cost=2.0759 ,acc=0.4508
t=480, eta=0.0599, loss=1.4839, cost=1.8810 ,acc=0.4831
t=640, eta=0.0799, loss=1.4661, cost=1.7858 ,acc=0.4880
t=800, eta=0.0999, loss=1.4556, cost=1.7188 ,acc=0.4912
t=960, eta=0.0801, loss=1.3650, cost=1.6095 ,acc=0.5321
t=1120, eta=0.0601, loss=1.2847, cost=1.5264 ,acc=0.5506
t=1280, eta=0.0401, loss=1.1689, cost=1.4159 ,acc=0.6116
t=1440, eta=0.0201, loss=1.1147, cost=1.3682 ,acc=0.6340
t=1600, eta=0.0001, loss=1.0836, cost=1.3393 ,acc=0.6499
t=1760, eta=0.0199, loss=1.1011, cost=1.3599 ,acc=0.6381
t=1920, eta=0.0399, loss=1.1225, cost=1.3883 ,acc=0.6169
t=2080, eta=0.0599, loss=1.1705, cost=1.4397 ,acc=0.6018
t=2240, eta=0.0799, loss=1.3199, cost=1.5906 ,acc=0.5512
t=2400, eta=0.0999, loss=1.3828, cost=1.6446 ,acc=0.5346
t=2560, eta=0.0801, loss=1.1911, cost=1.4575 ,acc=0.6009
t=2720, eta=0.0601, loss=1.1302, cost=1.4044 ,acc=0.6224
t=2880, eta=0.0401, loss=1.0601, cost=1.3427 ,acc=0.6622
t=3040, eta=0.0201, loss=1.0098, cost=1.3001 ,acc=0.6797
t=3200, eta=0.0001, loss=0.9788, cost=1.2717 ,acc=0.6953
t=3360, eta=0.0199, loss=0.9868, cost=1.2834 ,acc=0.6874
t=3520, eta=0.0399, loss=1.0205, cost=1.3245 ,acc=0.6652
t=3680, eta=0.0599, loss=1.0887, cost=1.3963 ,acc=0.6292
t=3840, eta=0.0799, loss=1.2509, cost=1.5562 ,acc=0.5604
t=4000, eta=0.0999, loss=1.7287, cost=2.0197 ,acc=0.4662
t=4160, eta=0.0801, loss=1.1548, cost=1.4465 ,acc=0.5998
t=4320, eta=0.0601, loss=1.0865, cost=1.3831 ,acc=0.6420
t=4480, eta=0.0401, loss=1.0266, cost=1.3309 ,acc=0.6684
t=4640, eta=0.0201, loss=0.9679, cost=1.2790 ,acc=0.6946
t=4800, eta=0.0001, loss=0.9378, cost=1.2510 ,acc=0.7141

Final test accuracy 0.4746 => 47.46%

2.3 Coarse search for lambda:

In the coarse search, I evaluated 8 logarithmically spaced values of λ in the range $l_{min} = 10^{-5}$ to $l_{max} = 10^{-1}$. A total of eight values were evaluated using a uniform grid in log-space. Each network was trained for 2 and 3 cycles using cyclical learning rates. The step size parameter was set $n_batch = 100$ to $n_s = 2 * np.floor(\frac{n}{n_batch})$. The three best performing parameters setting based on validation accuracy were:

- **Results with 2 cycles for the 3 best performing networks for Coarse search**

lambda = 0.00001, validation accuracy = 0.5254 = 52.54%
lambda = 0.00193, validation accuracy = 0.5218 = 52.18%
lambda = 0.00052, validation accuracy = 0.5206 = 52.06%

- **Results with 3 cycles for the 3 best performing networks for Coarse search**

lambda = 0.00014, validation accuracy = 0.5258 = 52.58%

lambda = 0.00001, validation accuracy = 0.5254 = 52.54%

lambda = 0.00052, validation accuracy = 0.5254 = 52.54%

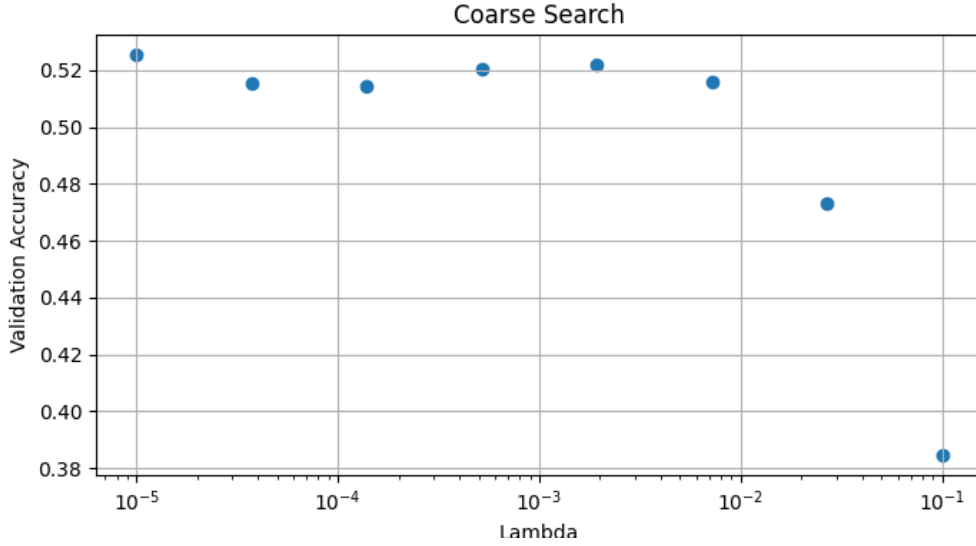


Figure 3: Validation accuracy for lambda values of coarse search with 2 cycles.

The results show that the best performing is obtained for small λ values between (10^{-5} to 10^{-3}) and larger λ values lead to worse performance, indicating excessive regularization.

2.4 Fine search for lambda:

After identifying a promising range of lambda values from the coarse search, a finer random search was performed to further optimize the regularization parameter. From the coarse search results, In the fine search, I sampled 8 additional λ values uniformly in log-space within the best-performing range $l_{min} = 10^{-5}$ to $l_{max} = 10^{-3}$. Thus, the fine search was conducted by sampling lambda randomly on a logarithmic scale within this interval. I used same dataset split as in coarse search.

Results with 2 cycles for the 3 best performing networks for fine Search:

lambda=0.00010, validation accuracy=0.5242 = 52.42 %

lambda=0.00170, validation accuracy=0.5232 = 52.32 %

lambda=0.00063, validation accuracy=0.5232 = 52.32 %

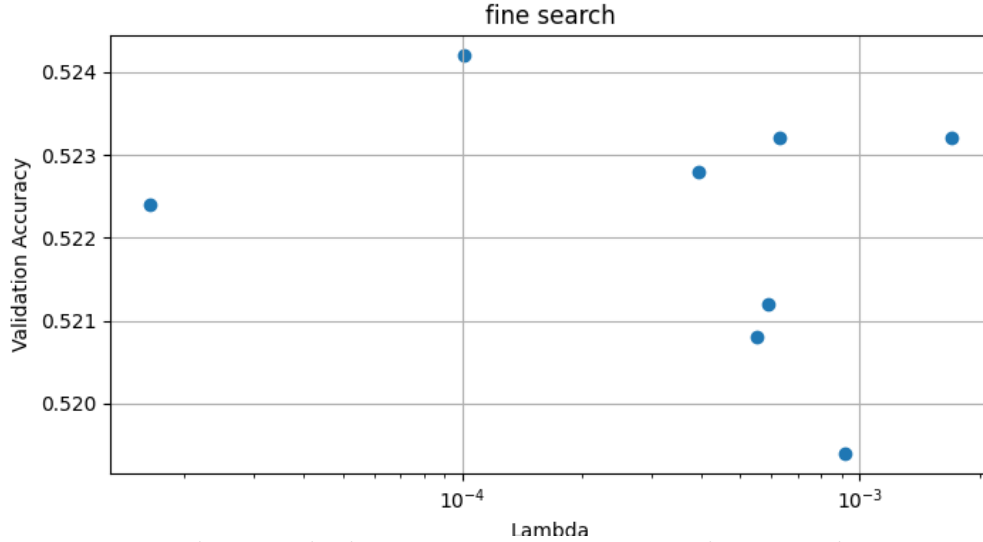


Figure 4: Validation accuracy for lambda values of fine search with 2 cycles.

Results with 3 cycles for the 3 best performing networks for fine Search:

lambda=0.00001, validation accuracy=0.5280 = 52.8 %

lambda=0.00030, validation accuracy=0.5264 = 52.64 %

lambda=0.00006, validation accuracy=0.5256 = 52.56 %

The fine search confirms that the optimal lambda lies within a narrow range around 10^{-5} and the validation accuracy stabilizes around 52% in 3 cycles and the best lambda found during the fine search with validation accuracy 52.8 %

2.5 Best lambda training:

The best value of the regularization parameter found from the validation set was $\lambda = 0.00001$. The network was trained using all available training data, while reserving 1000 examples for validation. Training was performed for approximately 3 cycles using cyclical learning rates with the following hyperparameters $n_batch = 100$, $n_s = 2 * np.floor(\frac{n}{n_batch})$ and $n_epochs = 12$.

cycle=1.0,t=196, eta=0.0199, loss=1.7909, cost=1.7915 ,acc=0.3730
 cycle=1.0,t=392, eta=0.0399, loss=1.6652, cost=1.6659 ,acc=0.4178
 cycle=1.0,t=588, eta=0.0599, loss=1.6079, cost=1.6085 ,acc=0.4350
 cycle=1.0,t=784, eta=0.0799, loss=1.5914, cost=1.5921 ,acc=0.4451
 cycle=1.0,t=980, eta=0.0999, loss=1.5850, cost=1.5857 ,acc=0.4366
 cycle=1.0,t=1176, eta=0.0801, loss=1.4860, cost=1.4868 ,acc=0.4786
 cycle=1.0,t=1372, eta=0.0601, loss=1.4196, cost=1.4204 ,acc=0.5002
 cycle=1.0,t=1568, eta=0.0401, loss=1.3703, cost=1.3711 ,acc=0.5194
 cycle=1.0,t=1764, eta=0.0201, loss=1.3098, cost=1.3107 ,acc=0.5432
cycle=1.0,t=1960, eta=0.0001, loss=1.2857, cost=1.2865 ,acc=0.5516
 cycle=2.0,t=2156, eta=0.0199, loss=1.3003, cost=1.3011 ,acc=0.5454
 cycle=2.0,t=2352, eta=0.0399, loss=1.3125, cost=1.3133 ,acc=0.5403
 cycle=2.0,t=2548, eta=0.0599, loss=1.3729, cost=1.3738 ,acc=0.5146
 cycle=2.0,t=2744, eta=0.0799, loss=1.5051, cost=1.5060 ,acc=0.4648
 cycle=2.0,t=2940, eta=0.0999, loss=1.5403, cost=1.5412 ,acc=0.4693
 cycle=2.0,t=3136, eta=0.0801, loss=1.3440, cost=1.3450 ,acc=0.5252
 cycle=2.0,t=3332, eta=0.0601, loss=1.3068, cost=1.3078 ,acc=0.5414
 cycle=2.0,t=3528, eta=0.0401, loss=1.2617, cost=1.2627 ,acc=0.5570
 cycle=2.0,t=3724, eta=0.0201, loss=1.2096, cost=1.2107 ,acc=0.5799
cycle=2.0,t=3920, eta=0.0001, loss=1.1844, cost=1.1855 ,acc=0.5877
 cycle=3.0,t=4116, eta=0.0199, loss=1.1987, cost=1.1998 ,acc=0.5817
 cycle=3.0,t=4312, eta=0.0399, loss=1.2262, cost=1.2273 ,acc=0.5702
 cycle=3.0,t=4508, eta=0.0599, loss=1.2610, cost=1.2621 ,acc=0.5565
 cycle=3.0,t=4704, eta=0.0799, loss=1.4397, cost=1.4409 ,acc=0.5039
 cycle=3.0,t=4900, eta=0.0999, loss=1.3749, cost=1.3761 ,acc=0.5169
 cycle=3.0,t=5096, eta=0.0801, loss=1.3077, cost=1.3090 ,acc=0.5402
 cycle=3.0,t=5292, eta=0.0601, loss=1.2312, cost=1.2325 ,acc=0.5649
 cycle=3.0,t=5488, eta=0.0401, loss=1.1860, cost=1.1873 ,acc=0.5795
 cycle=3.0,t=5684, eta=0.0201, loss=1.1462, cost=1.1475 ,acc=0.5973
cycle=3.0,t=5880, eta=0.0001, loss=1.1214, cost=1.1227 ,acc=0.6107

- Final Test accuracy with best lambda = 0.51200 = 51.2%
- Final Validation accuracy = 0.52100 = 52.2%
- Final Training accuracy = 0.61073 = 61.1%

After training, the final performance on the test set was 51.2% and figure 5 shows the training and validation loss plot of $\lambda = 0.00001$. The result is consistent with the expected performance of > 50% mentioned in the assignment.

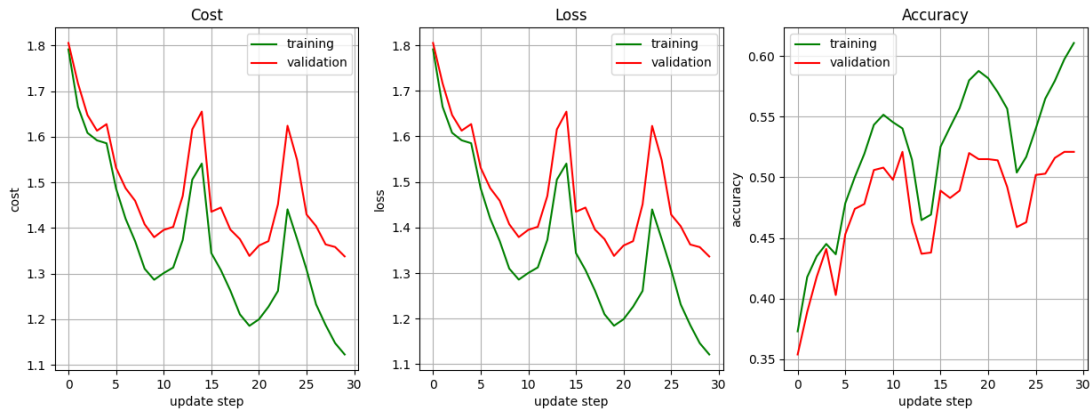


Figure 5: Training and validation loss, cost and accuracy with parameters setting $\text{lam}=0.00001$, $\text{cycles}=3$, $\text{eta_min} = 1\text{e-}5$, $\text{eta_max} = 1\text{e-}1$, $\text{n_epochs} = 12$ and $\text{n_batch} = 100$.

3. Discussion

The results show that cyclical learning rates improve the training process by allowing the model to explore different regions of the loss landscape. The oscillations observed in the loss curves correspond to changes in the learning rate and help avoid poor local minimum.

The coarse and fine search demonstrate that small value of λ lead to better performance, while larger values result in underfitting. The optimal range for λ is narrow, around 10^{-5} , which highlights the importance of careful hyperparameter tuning. Although the model achieves reasonable performance, the accuracy is limited by the simplicity of the two-layer architecture and the relatively short training time.

4. Conclusion

In this assignment, a two-layer neural network was successfully implemented and trained on the CIFAR-10 dataset. The correctness of the gradient computations was verified using PyTorch, showing small numerical difference.

Cyclical learning rates were applied effectively, improving convergence and performance. A systematic search for the regularization parameter λ identified an optimal value around 10^{-5} .

The final model achieved a test accuracy of **51.2%** which meets the expected performance for this assignment. With further hyperparameter extended training, the performance could likely be improved further.